

<h1 id="Laboratorio-3:-Detecci%C3%B3n-de-Anomal%C3%ADas-con-Isolation-Forest">Laboratorio 3: Detección de Anomalías con Isolation Forest</h1><h2 id="Paso-1:-Importar-librer%C3%ADas">Paso 1: Importar librerías</h2>

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
import joblib
```

<h2 id="Paso-2:-Cargar-y-explorar-datos-(EDA)">Paso 2: Cargar y explorar datos (EDA)</h2>

```
In [2]: # Cargar datos de ejemplo (simulando tráfico de red)
np.random.seed(42)
n_normal = 1000
n_anomalias = 50

# Datos normales
datos_normales = pd.DataFrame({
    'duracion': np.random.normal(10, 2, n_normal),
    'bytes_enviados': np.random.normal(500, 100, n_normal),
    'paquetes': np.random.normal(20, 5, n_normal),
    'puerto_destino': np.random.choice([80, 443, 22], n_normal)
})

# Datos anómalos
datos_anomalos = pd.DataFrame({
    'duracion': np.random.normal(60, 10, n_anomalias),
    'bytes_enviados': np.random.normal(5000, 1000, n_anomalias),
    'paquetes': np.random.normal(200, 50, n_anomalias),
    'puerto_destino': np.random.choice([8080, 9090, 6667], n_anomalias)
})

# Combinar datos
datos = pd.concat([datos_normales, datos_anomalos], ignore_index=True)
datos['etiqueta'] = [0]*n_normal + [1]*n_anomalias

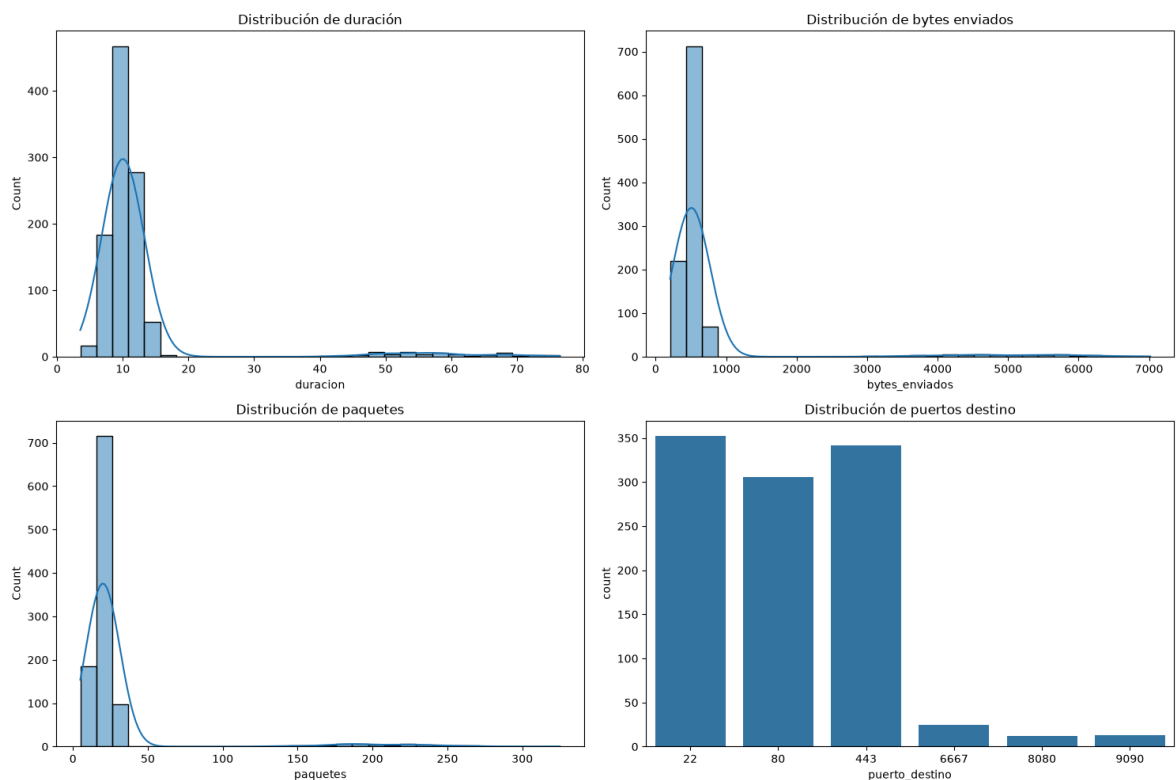
print(f"Datos cargados: {len(datos)} registros")
print(f"Normales: {n_normal}, Anomalías: {n_anomalias}")
datos.head()
```

Datos cargados: 1050 registros
Normales: 1000, Anomalías: 50

```
Out [2]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }
.dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; }
</style> <thead> <th></th> <th>duracion</th> <th>bytes_enviados</th>
<th>paquetes</th> <th>puerto_destino</th> <th>etiqueta</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>
```

10.993428	639.935544	16.624109	443	0
9.723471	592.463368	19.277407	443	0
11.295377	505.963037	16.037900	443	0
13.046060	435.306322	18.460192	80	0
9.531693	569.822331	10.531927	22	0

```
In [3]: # EDA: Histogramas
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
sns.histplot(datos['duracion'], bins=30, ax=axes[0,0], kde=True)
axes[0,0].set_title('Distribución de duración')
sns.histplot(datos['bytes_enviados'], bins=30, ax=axes[0,1], kde=True)
axes[0,1].set_title('Distribución de bytes enviados')
sns.histplot(datos['paquetes'], bins=30, ax=axes[1,0], kde=True)
axes[1,0].set_title('Distribución de paquetes')
sns.countplot(x='puerto_destino', data=datos, ax=axes[1,1])
axes[1,1].set_title('Distribución de puertos destino')
plt.tight_layout()
plt.show()
```



Paso 3: Preprocesamiento de datos

```
In [4]: # Seleccionar características
X = datos[['duracion', 'bytes_enviados', 'paquetes', 'puerto_destino']]
```

```
y = datos['etiqueta']

# Normalizar datos
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Datos normalizados")
```

Datos normalizados

<h2 id="Paso-4:-Entrenar-modelo-Isolation-Forest">Paso 4: Entrenar modelo Isolation Forest</h2>

```
In [5]: # Entrenar modelo
modelo = IsolationForest(contamination=0.05, random_state=42)
modelo.fit(X_scaled)

# Predecir
predicciones = modelo.predict(X_scaled)
predicciones = np.where(predicciones == -1, 1, 0) # Convertir -1 (anomalía) a 0

print("Modelo entrenado")
```

Modelo entrenado

<h2 id="Paso-5:-Evaluar-modelo">Paso 5: Evaluar modelo</h2>

```
In [6]: # Métricas
print("Matriz de confusión:")
print(confusion_matrix(y, predicciones))
print("\nReporte de clasificación:")
print(classification_report(y, predicciones))
```

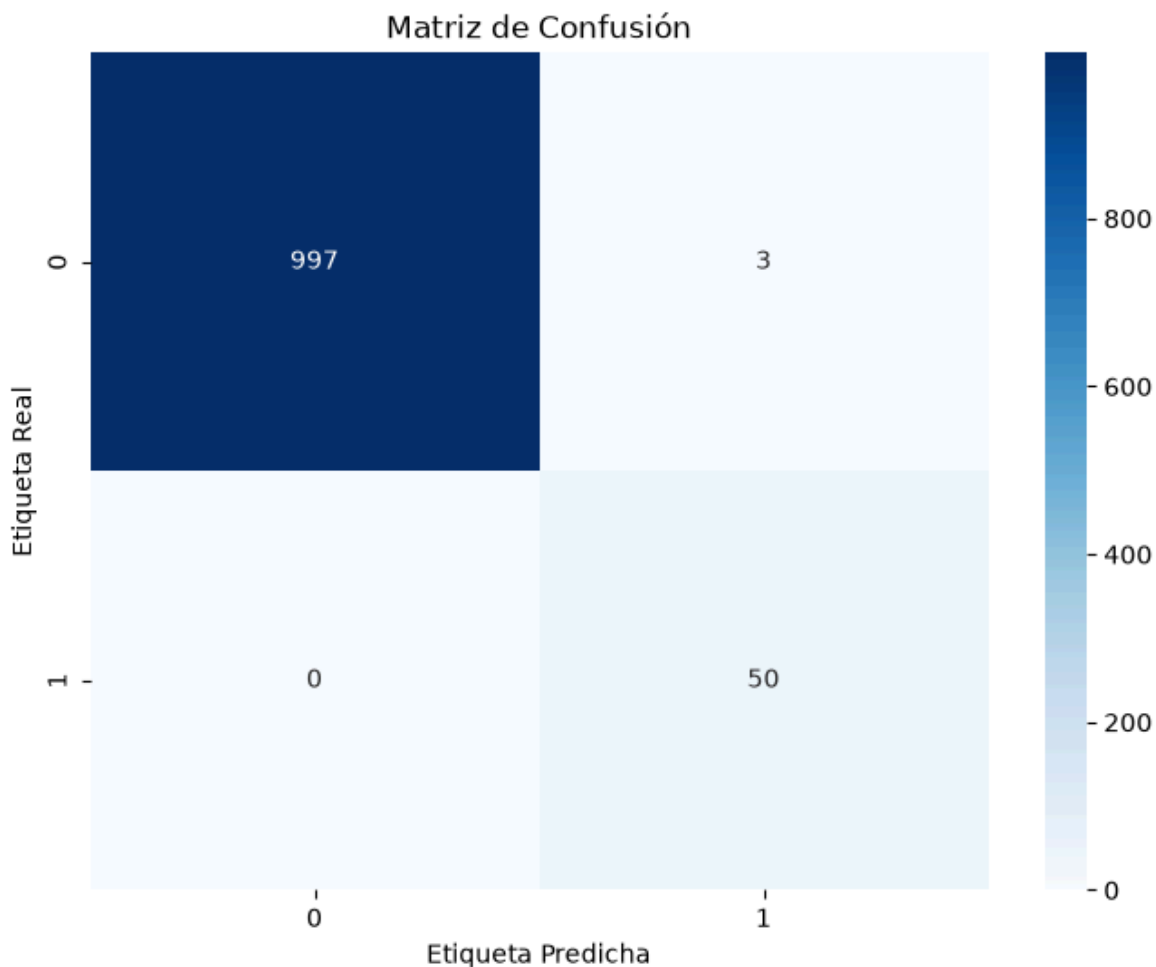
Matriz de confusión:

```
[[997  3]
 [ 0 50]]
```

Reporte de clasificación:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1000
1	0.94	1.00	0.97	50
accuracy			1.00	1050
macro avg	0.97	1.00	0.98	1050
weighted avg	1.00	1.00	1.00	1050

```
In [7]: # Visualizar matriz de confusión
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y, predicciones), annot=True, fmt='d', cmap=
plt.title('Matriz de Confusión')
plt.ylabel('Etiqueta Real')
plt.xlabel('Etiqueta Predicha')
plt.show()
```



Paso 6: Analizar umbrales y F1-Score

```
In [8]: # Obtener scores de anomalía
scores = modelo.decision_function(X_scaled)
datos['score'] = scores

# Probar diferentes umbrales
from sklearn.metrics import f1_score
umbrales = np.linspace(np.min(scores), np.max(scores), 100)
f1_scores = []

for umbral in umbrales:
    pred_umbral = np.where(scores < umbral, 1, 0)
    f1_scores.append(f1_score(y, pred_umbral))

# Encontrar mejor umbral
mejor_indice = np.argmax(f1_scores)
mejor_umbral = umbrales[mejor_indice]
mejor_f1 = f1_scores[mejor_indice]

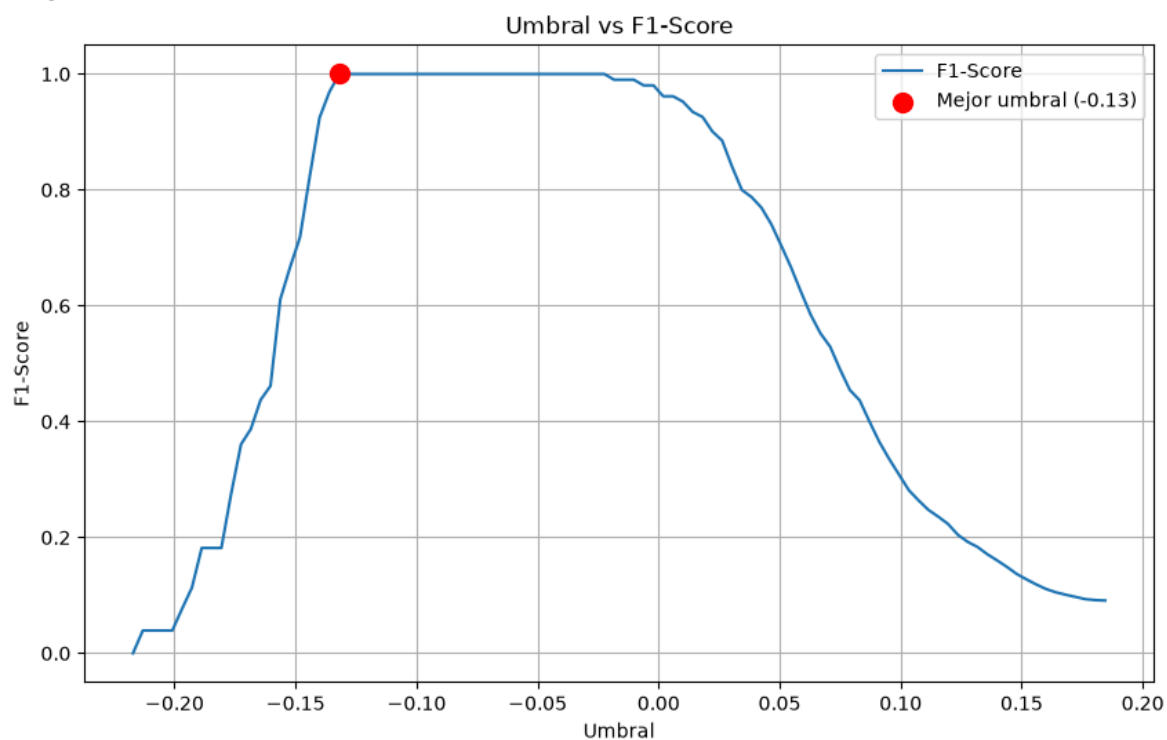
print(f"Mejor umbral: {mejor_umbral:.4f}")
print(f"Mejor F1-Score: {mejor_f1:.4f}")

# Visualizar
plt.figure(figsize=(10, 6))
plt.plot(umbrales, f1_scores, label='F1-Score')
plt.scatter(mejor_umbral, mejor_f1, color='red', s=100, zorder=5, label=f)
plt.xlabel('Umbral')
```

```
plt.ylabel('F1-Score')
plt.title('Umbral vs F1-Score')
plt.legend()
plt.grid(True)
plt.show()
```

Mejor umbral: -0.1320

Mejor F1-Score: 1.0000



```
In [9]: # Top 10 anomalías
top_anomalias = datos.sort_values('score').head(10)
print("Top 10 anomalías:")
top_anomalias
```

Top 10 anomalías:

```
Out [9]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }
.dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; }
</style> <thead> <th></th> <th>duracion</th> <th>bytes_enviados</th>
<th>paquetes</th> <th>puerto_destino</th> <th>etiqueta</th> <th>score</th>
</thead> <tbody> <th>1019</th> <th>1010</th> <th>1009</th> <th>1021</th>
<th>1006</th> <th>1000</th> <th>1007</th> <th>1027</th> <th>1004</th>
<th>1023</th> </tbody> <tbody></tbody>
```

72.264222	6659.095521	252.559189	9090	1	-0.217146
67.433121	7011.825996	181.259781	9090	1	-0.198348
65.198634	6273.674248	269.216221	6667	1	-0.196288
75.229806	6480.108792	220.442451	8080	1	-0.191672
67.355243	4616.538954	296.310378	6667	1	-0.189120
49.595450	6176.856244	243.297634	9090	1	-0.180507
76.584850	4596.632327	147.608897	6667	1	-0.179500
57.386372	3071.269977	325.189811	8080	1	-0.178495
70.741854	4172.651000	179.919614	9090	1	-0.174986
45.930004	5785.125221	268.945085	6667	1	-0.173488

Paso 7: Guardar modelo

```
In [10]: # Guardar modelo y scaler
joblib.dump(modelo, 'modelo_anomalias.pkl')
joblib.dump(scaler, 'scaler.pkl')
print("Modelo y scaler guardados exitosamente")
```

Modelo y scaler guardados exitosamente

```
In [ ]:
```